Inputs, outputs, and readouts

ar087 · 14 August 2026 · Draft

Contents

1. Developer guide
2. Outputs and observables
3. Standard readouts
4. Input bindings
5. API reference

Developer guide

Outputs and observables

An output is a named value returned by the graph. An observable exposes an internal signal for recording without changing the computation.

```
readout = snn.readouts.MeanVoltage(
    source=cell.E.spikes,
    classes=10,
    name="classifier",
)
net.output("class_logits", readout)
net.expose(cell.E.spikes, cell.I.spikes, name="hidden")
```

Standard readouts

The standard readouts are mean voltage, final voltage, spike count, duration-normalized spike rate, and cumulative potential. Spike-rate logits divide spike count by valid presentation duration, so changing duration does not silently rescale the classifier.

All five standard readouts execute in the graph executor. Spike rate reports spikes per second from either an explicit duration in seconds or a `(time, batch)` valid-time mask; masked reductions reject empty windows rather than silently returning arbitrary values. The compiler infers shapes and units, checks linear readout parameter shapes, rejects malformed masks and ambiguous durations, and records operation requirements in the bundle capability manifest.

Input bindings

The execution layer binds concrete data to graph inputs without placing datasets or stimuli in graph structure. Dense arrays and sparse event streams are implemented as separate named, data-only representations. Dense NPY replay binds to a graph with one input; dense NPZ arrays bind by input identifier.

Before execution, the resolver requires exact input coverage, common time and batch axes, declared feature dimensions, finite numeric values, binary spike values, and boolean or zero/one masks. A mismatch names the offending input and fails before simulation.

Event replay stores zero-based integer step, batch, and channel coordinates plus explicit step and batch counts. Coordinates must be ordered by step, batch, and channel. The resolver rejects duplicates, invalid graph input types, inconsistent durations, and out-of-bounds coordinates before materializing binary spikes.

Every replay emits a versioned execution protocol containing the representation, source-file digest and array keys, resolved shape and data type, signal type and unit, dataset identity and split when supplied, sample cap, batch size, shuffle behavior, timestep, duration, masks, seeds, and representation-specific resolution semantics. The command-line contract accepts `--input-file` or `--event-file`, dataset metadata, and the explicit `--input-shuffle` or `--no-input-shuffle` pair.

Fixed-rate and categorical variable-rate Poisson encoding belong to execution protocol rather than graph topology. Both are implemented as seeded bindings. The categorical protocol samples one rate uniformly and independently per presentation, records the realized rate vector, and then generates graph-shaped spikes using the declared timestep.

Dense-array, valid-time-mask, sparse event-stream, generated Poisson, and portable dataset-snapshot bindings work today. A snapshot remains an external immutable NPZ file identified by its content digest and caller-supplied dataset/split identity. The standard encoder vocabulary covers feature-scaled rate-Poisson samples, already binned spikes, and timestamped event samples binned at the graph timestep.

API reference

Outputs and observables

```
net.output(name, signal) -> SignalLike
net.expose(*signals, name=None) -> None
```

An output maps a public name to one graph signal. `expose` records one or more internal signals. A single exposed signal uses `name` directly; multiple signals use `<name>_0`, `<name>_1`, and subsequent indices.

Standard readouts

```
snn.readouts.MeanVoltage(*, source, classes, name, tau=2 * snn.ms,
weight=Normal(1.0, 0.1))
snn.readouts.FinalVoltage(*, source, classes, name)
snn.readouts.SpikeCount(*, source, classes, name)
snn.readouts.SpikeRate(*, source, classes, name, duration=None, mask=None,
window="full")
snn.readouts.CumulativePotential(*, source, classes, name)
```

Every constructor returns a `Readout`, which delegates signal attributes and exposes its parameter identifiers. `classes` is the output width. `MeanVoltage` defaults to the legacy COBANet output membrane's 2 ms time constant; alternatives remain explicit authoring

choices. `SpikeRate` requires either `duration` in seconds or a `(time, batch)` mask. The only implemented window is `"full"`.

Dense bindings

```
DenseArrayBinding(input_id, value, source={})
load_dense_array_bindings(path, graph) -> tuple[DenseArrayBinding, ...]
resolve_dense_array_bindings(
    graph, *, bindings=(), inputs=None,
    device="cpu", seed=0, protocol=None,
) -> ResolvedDenseInputs
```

`value` is a PyTorch tensor. `source` is JSON-serializable provenance. NPY binds only to a graph with one input; NPZ keys bind by graph input identifier. Direct `ExecutionSpec.inputs` values are converted to in-memory bindings and validated through the same resolver.

The resolved protocol uses schemas `tools/snn.dense-array-binding/v1` and `tools/snn.execution-protocol/v1`. Reserved protocol fields cannot be overridden by caller metadata.

Event-stream bindings

```
EventStreamBinding(
    input_id, steps, batches, channels,
    steps_count, batch_size, source={},
)
load_event_stream_bindings(path, graph) -> tuple[EventStreamBinding, ...]
resolve_event_stream_bindings(
    graph, *, bindings,
    device="cpu", seed=0, protocol=None,
) -> ResolvedDenseInputs
resolve_input_bindings(
    graph, *, dense_bindings=(), event_bindings=(), inputs=None,
    device="cpu", seed=0, protocol=None,
) -> ResolvedDenseInputs
```

Event bindings support graph inputs with shape `("time", "batch", channels)` and signal type `"spikes"`. Coordinate arrays are one-dimensional integer tensors of equal length. Step coordinates lie in `[0, steps_count)`, batch coordinates in `[0, batch_size)`, and channel coordinates in the declared graph width.

A single-input NPZ uses `steps`, `batches`, `channels`, `steps_count`, and `batch_size`. Multi-input files prefix each key with the input identifier and a period. Typed requests may combine event-stream spike inputs with dense masks or continuous inputs when their time and batch axes agree.

The event schema is `tools/snn.event-stream-binding/v1`; a mixed request uses `tools/snn.mixed-input-bindings/v1`. The protocol records ordering, duplicate rejection, binary materialization, event counts, masks, and file provenance.

Dataset snapshots and encoders

```
DatasetEncoder(
  kind, duration_ms=None, max_rate_hz=None, seed=0,
)
DatasetSnapshotBinding(
  path, input_id, dataset_id, split, encoder,
  target_id=None,
  feature_key="features", label_key="labels",
  sample_cap=None, shuffle=False, order_seed=0,
)
resolve_dataset_snapshot_binding(
  graph, binding, device="cpu", execution_seed=0,
) -> tuple[ResolvedDenseInputs, tuple[TargetArrayBinding, ...]]
```

The binding uses schema `tools/snn.dataset-snapshot-binding/v1` and currently requires a single spike input shaped `(time, batch, channels)`. Every snapshot contains one-dimensional integer labels. A target id turns the selected labels into a digest-bearing training target; simulation may omit the target id.

`rate_poisson` expects floating `features` shaped `(samples, channels)` with finite values in `[0, 1]`. Each value scales `max_rate_hz`; seeded Bernoulli discretization produces the declared physical duration at graph `dt`. The maximum rate must keep per-step probability at most one.

`prebinned_spikes` expects binary `features` shaped `(time, samples, channels)` and preserves the exact time axis. `event_bin` expects equal-length `event_sample` and `event_channel` integer arrays plus floating `event_time_ms`. It applies left-closed, right-open floor bins, unions collisions into binary spikes, and records retained-event and collision counts.

Selection starts from snapshot sample indices, applies a seed-derived permutation when requested, and then applies the cap. The execution protocol records the snapshot path and SHA-256, dataset identity, split, total and selected sample indices, label/feature keys, batch size, shuffle flag, order seed, encoder parameters and seed, graph timestep, physical duration, and resolved target provenance.

The CLI selects this route with `--dataset-file`, `--dataset-encoder`, `--input-dataset-id`, and `--input-split`. Training also supplies `--dataset-target-id`; `--max-samples`, `--input-shuffle`, `--seed`, `--t-ms`, and `--input-rate` define selection and encoding where applicable. Dataset snapshots cannot be combined with replay or generated-input bindings.

Next: Training recipes and graph-native learning